Yazılabilirlik: Kaç satır kodda yazıyorsun. Az ise güzel

Güvenilirlik: Kodu ilk yazdığında direkt çalıştıracak mı acaba

Aranızda "Alt Programlar" a kadar dahil edecekmiş

Slayt Konu 10, 15. sayfaya kadar işledi

+ Yan dayt yan. boşluk doldurma yan kişiği olamamış

Dinamik ve Statik Scope'a ancak vaktin artan diğe kişiği son dakikaymış

Static scope kullanıldığında değişkenin değeri ..... diye yeni boşluk doldurma

Dilbe ara parantez sağdan kaldı (10. slayt bitiyo yani)

Aşağıdaki kodun hangi operatör önceliği gerektirir?

" " dynamic type binding örnektir

" programda static scope kullanırsam x'in değeri ne dir, dinamic kullanırsam ne olur?

floating point: Şu bildiğin float. Virgüllü sayılar.

exponent (üs) → 2^3

Tarihsel Sıralama:

1-) Makine ve Assembly dili: Bayt, okunabilir, düşük seviye bir dil olan assembly geliştirilmiş.

2-) İlk yüksek seviyeli diller: Fortran (bilimsel hesaplamalar), COBOL (ticari işlemler), Lisp (yapay zeka)

Daha insana yakın dillerdir. Amaç donanıma bağlılığı azaltmaktır.

3-) Yapısal ve Sistem dilleri: "Structured programming" kavramı doğdu. C, Pascal, Ada

4-) Nesne ve internet çağı: OOP (Object oriented programming) C++, Java, Python, Javascript

5-) Modern ve özel amaçlı diller: Go, Rust, Kotlin, Swift, TypeScript, Haskell

Markup dil: Programlama dili gibi işlem yapmaz. Etiket (tag) kullanır. XML, HTML gibi.

Döngü, if-else, hesaplama yoktur programlama dillerindeki gibi. Veri saklama, tanımlama ve Web içindir.

(diklik)

Ortogonallik: Bir özellik diğerine bağlı değilse ortogonaldir. Mesela: int* float* char* hepsi pointer. Her tipi gösterebiliyor. Örneğin "list, sadece int tutabilir" falan derseydi ortogonal olmazdı.

Burda pointer ve primitive (basit, int) tipler birbirine diktir.

Imperative (emredici) diller: Bilgisayara nasıl nasıl yapacağını söyler. C, C++, Java, Fortran

Fonksiyonel diller: Nasıl değil "ne" istediğini söylersin. Haskell, LISP

Nesne Yönelimli (Object-Oriented): Java, C++

Mantıksal (Logic Programming): Kurallar + gerçekler. Prolog

Prosedural (Yordam Temelli): Imperative'in alt türü. C, Pascal

---

Syntax → söz dizim. Kodun nasıl yazıldığı.

Semantik → anlam. Yazdığın kodun ne anlama geldiği

Programlama Dillerinde Quality Attributes

1) **Okunabilirlik:** Kodun ne kadar kolay anlaşılabildiği

2) **Yazılabilirlik:** // Kolay yazılabilmesi. Örn: GUI için Visual Basic işletim sistemi için C

3) **İfade edilebilirlik:** Aynı işi daha kısa ve pratik şekilde yapabilme. Örn: count++

4) **Ortogonallik:** Bağımsız ve serbestçe kombinlenebilirlik. Örn: Liste her türü tutabiliyorsa, pointer her tipi gösterebiliyorsa

5) **Sadelik (Simplicity):** Karmaşıklıktan uzak olması. Ortogonallik arttıkça sadelik artar! Fonksiyonel diller (Lisp gibi)

6) **Güvenilirlik:** Doğru ve hatasız çalışabilmesi.
   - a) Tür denetimi (Type checking): Programdaki tür hatalarının ya derleyici tarafından ya da çalışma sırasında tespit edilmesi. Run-time denetimi maaliyetli olduğundan compile-time tür denetimi tercih edilir.
   - b) Aliasing problemi: Aynı belleğe birden fazla isimle erişim hata riskini artırır.

7) **Maaliyet:** Öğrenme maliyeti, yazma m., Derleme m., Çalıştırma m.

Program Çalıştırma

1- **Derleyici (Compiler)** Tamamen makine koduna çevirir, hızlı çalışır. C, COBOL, C++

2- **Yorumlayıcı (Interpreter)** Kod satır satır okunur ve anında çalıştırılır. Python

3- **Ara dil (hibrit) (Bytecode)** Kod önce evrensel bir ara dile çevrilir, sonra sanal makine çalıştırılır. Java

4- **JIT (Just-in-time)** Bytecode olarak başlar, sonra sık kullanılan fonksiyonları makine koduna çevirir. Sonra hızlanır. C#

Saf Yorumlama: Kerim yapılmıştır. Program doğrudan yorumlayıcı tarafından çalıştırılır. JavaScript

Linker: Programın nihai halini verir. Derleyici tarafından üretilen kodları ve farklı kütüphaneleri bir araya getirir.

Prolog: Bir veritabanı gibi çalışır, ama sadece veri değil, kurallar da içerir.

a) **Facts (olgular):** Doğru kabul edilen bilgiler. mother(mia, jake). father(john, mia).

b) **Rules:** Yeni bilgi üretmek için kullanılır. grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

c) **Goal:** Sisteme son soru sorarsın. father(bob, dorcie). Bob, Dorcie'nin babası mı?

X, Y'nin ebeveyni VE Y, Z'nin ebeveyniyse → X, Z'nin büyükanne/babasıdır.

Sonra doğru true veya false döndürür.

* Java'da pointer yokmuş. C++ gibi ama daha güvenliymiş.

Halting (Durma) Problemi

Herhangi bir P programı ve x girdisi verildiğinde, P(x)'in sonlanıp sonlanmayacağını her zaman doğru bir şekilde belirleyen bir program yazılamaz. Eğer böyle bir program olsaydı, sonsuz döngüye giren programları çalıştırmadan anlayabilirdik. Ve yazılım hatalarını otomatik olarak tespit edebilirdik. Ama bu imkansızdır.

Örneğin: L dili diye bi programlama dili olsun. Sonra P diye bi program yazılsın ve x onun girdisi olsun. Eğer P(x) sonlanıyorsa sonlanıp "yes" yazdıran, P(x) döngüye giriyorsa yine sonlanıp "no" yazdıran bir H programı var mıdır?

Böyle bir H programının olmadığını çelişki yöntemiyle yapıyoruz.

Yani bi K programı tanımlıyoruz. K(P) → eğer H(P,P) == "no" ise "yes" yaz ve dur
eğer H(P,P) == "yes" ise sonsuz döngüye gir.

Yani H(P,P) → P programı kendi girdisiyle çalışırsa durur mu? Diğerleri yani

Hesaplanabilirlik

Karar verilemezlik (Undecidability): Programların bazı özelliği algoritmik/mekanik yöntemlerle belirlenemez. Halting problemi bunlardan biridir.

Hesaplanabilirlik (Computability): Bir fonksiyon, onu hesaplayan bir program varsa hesaplanabilirdir. Halting'in karar verilemezliği, hesaplanamaz fonksiyonların var olduğunu gösterir.

Kısmi tanımlılık (Partiality): Bazı fonksiyonlar bazı girdilerde tanımsız olabilir.

Turing-Tamlık (Turing Completeness): Bir sistem Turing Makinesi kadar güçlü ise Turing tamdır. Döngü, koşul, bellek destekliyorsa teoride her şeyi hesaplayabilir. C, Python, Java gibi.

Tanıma (Recognition): Mesela bir string alır, bu dile ait mi değil mi bilir.

Üretim (Generation): "Bir dil: hangi kurallarla oluşturabiliriz"

Gramer: Dilin kuralları. Noam Chomsky'e göre 4 farklı gramer kuralı vardır:

1-) Regular Grammar: Düzenli gramer, basit yapılar için. Token (kelimelerde) kullanılır.

2-) Context Free Grammar: bağlam bağımsız gramer. Her yerde aynı şekilde uygulanır.

BNF (Backus-Naur Form): Programlama dilinin kurallarını yazmak için kullanılan özel bir gösterim.

Başka bir dil tanımlayan bir dil olduğu için metalanguage' dir.

Örnek

Verilen gramer kullanarak aşağıdaki deyimler için sol türetim ve türetme kullanarak parse tree çizelim.

atama

<assign> → <id> = <expr>

<id> → A | B | C

<expr> → <id> + <expr>

| <id> * <expr>

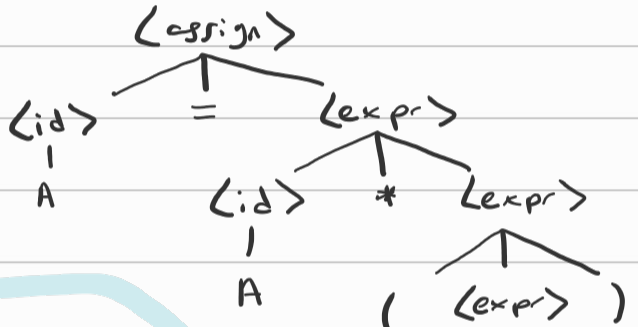
| (<expr>)

| <id>

↓ expression

a) A = A * (B + (C * A))

parse tree:



Değer işlem sırası belirlenmediğinde belirsizliği (ambiguity) ortadan kaldırmak için bunu gibi bir gramer yazılabilir.

Bunun için ayrı nonterminal semboller kullanılır.

A = B + C * A

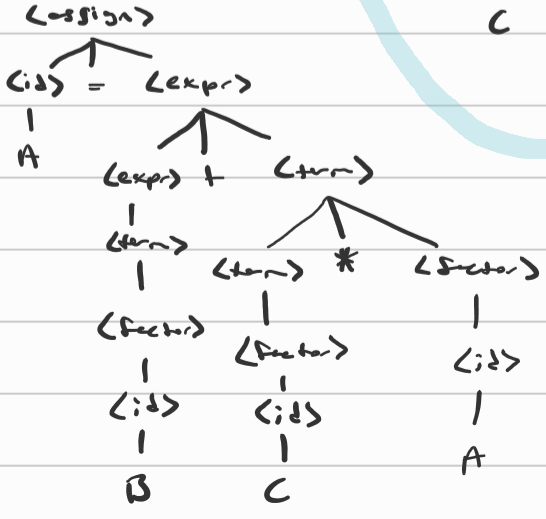
<assign> → <id> = <expr>

<id> → A | B | C

<expr> → <expr> + <term> | <term> (toplama işlemleri)

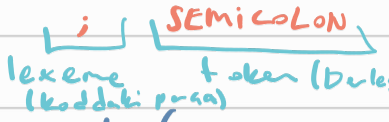
<term> → <term> * <factor> | <factor> (çarpma işlemleri)

<factor> → (<expr>) | <id> (öncelikleri belirtir)



Derleyici bi kodu aldığı zaman önce bi sözcüksel analiz (lexical) yapar. Sonra bi sözdizimsel analiz (Syntax), sonra semantik, kod üretimi falan

Lexeme: Bir dilin en alt seviyedeki syntactic birimdir. (*, sum, begin) Atom gibi



Lexical (Sözcüksel Analiz): Metni küçük parçalara (token) ayırır. Sonra Syntax (Sözdizimsel) Analysis ile tokenları doğru sırada mı bakılır.

Bu ikisi 3 sebepten ayrıdır:

1) Basitlik
2) Verimlilik
3) Taşınabilirlik

Parser (Sözdizimcisi)

Ağaçları oluşturmaya yarar.

Kökten yapraklara (Top-down): Ağaç kökten yapraklara doğru inşa edilir

Yapraklardan köke (Bottom-up): Yapraklardan köke

Scope Kuralları (Görünürlük)

Static (Lexical) Scope: Compile-time, program metnine bakar, yaygın

Dynamic Scope: Runtime, son aktivite bağlama bakar, nadir

Statik Scope Örneği:

```
x = 10
def f():
    print(x)
def g():
    x = 20
    f()
g()
```

Çıktı 10 dur.

x değeri sadece o fonksiyonun içinde geçerlidir.

Dinamik Scope Örneği:

aynı kodda çıktı

20 olur.

Fonksiyon nerede çağırıldığına bakılır.

Tür Dönüşümleri

Widening: int → float (güvenlidir)

Narrowing (Daraltmalı): double → float (veri kaybı olabilir!)

Mixed-Mode Expression: Farklı türde iki sayı. int a = 5; float b = 2,5; float d = b * a;

Coercion (Örtük dönüşüm): Yukarıdaki gibi a'yı da otomatik float'a çevirmesi ve yapması

Pass-by-Value: Değer kopyalanır. Fonksiyon içinde değişen x ile dışarıdaki x farklıdır.

Pass-by-Reference: Adres gönderir. Fonksiyon dışında bile x'teki değişiklik olur.

Fortran: Sadece bilimsel hesaplamalar ve floating point işlemleri için geliştirilmiştir. İlk yüksek seviyeli dildir.

Algol: Fortran'la aynı anda daha gelişmiş. Blok yapısını ilk destekleyen program. Aynı zamanda Nested fonksiyonlarda.

Cobol: Bankacılık

LISP: Yapay zeka için ilk yüksek seviyeli fonksiyonel prog. dilidir. Dinamik veri yapısı dan Liste'ler ilk kez kullanıldı.

Prolog: Yapay zeka " " " sembolik programlama dilidir.

Pascal:

Perl:

Haskell: Bir değişkenin değeri sabit kalmalı tanımlandıktan sonra değiştirilemez :O

Scala

Go

Rust

Swift

Ada: AND yerine and then yazdığında bu dilde bilgisayar iki tarafı da çalıştırmaz. Mesela VE'nin bir tarafı yanlışsa diğerine hiç bakmıyor çünkü zaten sonuç yine yanlış olacak.

Buna Short Circuit (Kısa devre) denir. Bu özellik yazılabilirliği artırır.

Bir dizi (array) elemanına indeksle erişimde indeksler genelde int olur, ama ADA'da bu Boolean, char, enum da olabilirmiş.

Erlang: Telekomünikasyon. Atomik değişkenler değiştirilemez. for, while döngüleri Recursion ile yapılıyor.

Gramer Örn Soru:

<S> → <A> a <B> b (sonda en az 1 tane b olmalı)

<A> → <A> b | b

<B> → a <B> | a

Hangi kelimeler verilen bu gramerden türetilmiştir?

a) baab

b) bbbab (<B>den gelen bir tane a eksik)

c) bbaaaa

d) bbaab

Binding (Bağlama): Bir isim ile onun temsil ettiği nesne arasındaki ilişki. (int wow;)

Aliasing: Takma ad. Aynı nesnenin aynı ortamda birden fazla isimle erişilmesi.

Scope için örnek:

```
var x; // global
function sub1(){
    var x;  // sub1 scope
    function sub2(){
        ....
    }
}

function sub3(){
.....
}
```

en yakın scope'lar kullanılır:

sub1 → sub1 içindeki var x'i kullanır.

sub2 → sub1 içindeki var x'i kullanır. Kendisinde yok ancak. Ona en yakın enclosing scope da sub1'de

sub3 → Global var x'i kullanır. Kendi x'i yok ancak

Record: Array gibi ama farklı tipler bir arada tutulabiliyor. Elemanlar isimleriyle erişilir. Nested olabilir. (Record içinde başka record olabilir)

Farklı dillerde:

C/C++/C# → Struct

Java → class (data class)

Python/Ruby → dictionary (hash)

Strongly Typed: Veri tipleri ve dil kurallarında aşırı katı olmaması.
Mesela kafana göre elma ile armutu toplayamamasın. "5" ile 10'u toplayamamasın.

Ada: Neredeyse strongly typed.
ML, F#, Scala: Tam anlamıyla strongly typed.

Coercion → Otomatik tip dönüşümü. int ile float'ı toplamak gibi. Strong typing'i zayıflatır.

C, C++ → Strongly typed değil.

Java, Scala, C# → Strong typing var ama sınırlı coercion

Binary Atama: a+1 mesela **Unary Atama**: a++

⭐ GEMİNİ SOHBET GEÇMİŞİNE BAK test ve kod örnekleri var.

Saf Fonksiyon: Sadece sabit bir değer döndürüyor. Hiçbir yan etkisi yok.

Statik Değişken: Hafızadan silinmeyen değişkenler. Mesela bir fonk. içine `static int sum = 0;` dedik. Fonksiyon bi' daha çağrıldığında o değişken yeniden oluşturulmuş.

Python → fonksiyon içi değişkenin global olmasını istiyorsan başına global yazmalısın.
Lua → Python'ın tam tersi. Değişkenin yerel yapmak için local demen lazım.

Pass-by-value: in mode uygular. Aktüel ile, biçimsel parametre bağımsızdır.
actual parameter (argüman/aktüel) → hesapla(x)
biçimsel (fonksiyondaki) → void hesapla(int x)

Pass-by-result: out mode. Biçimsel parametrenin değeri actual parametreye kopyalanır.

Pass-by-Value-result: inout mode. ikisinin birleşimidir, (copy-in, copy-out)

⭐ Fonksiyon içi fonksiyon yazarken parametre sayısı ve veri tipleri uyuşmalı, return tipi fark etmez. C'de function pointer, C#'ta da Delegate kullanılır.

Bir x değişkeninin Nested fonksiyon içinde 3 farklı yerde tanımlandığını düşün. Ekrana ne basacağın binding'lerle ilgili: Mesela x sırasıyla 10, 20, 30 tanımlı.

1-Deep Binding: Fonksiyonun ilk tanımlandığı yer baz alır. Modern dillerin çoğu bunu kullanır. Static scope ile uyumludur.

2- Shallow Binding (Sığ bağlama): Fonksiyonun bittiği çağrıldığı yerin ortamını baz alır. Dinamik scope ile uyumludur. Önerilmez. Zor ve nadiren kullanılır.

3- Ad Hoc Binding:

Parametrik Polimorfizm: Bir fonksiyona farklı türde parametreler gönderebilir. İstersem int de yollarım float da vs.

C dilinde generics yerine makrolar kullanılmış Ama ciddi hatalar alabilir

Closure: Değişkenler olmasın diye stack yerine Heap'te tutulur. Static scope olmalı ve nested fonksiyon yapısı olmalı ama

Coroutine: Bir işi yapıken resume edip sonra kaldığı yerden devam edebilen fonksiyon.